

Leave this box blank
Please submit online <http://joiv.org/index.php/joiv/login> in DOC file

Editor will not receive submission by email

Please be sure to check for spelling and grammar before submitting your paper.

Machine Learning-Based Prediction of High-Performance Concrete Strength: A Comparative Study of Random Forest, XGBoost, and LightGBM

Haniya Harum Pekerti Regar¹, Nadia Elvina Prameswari², Ayunda Fatikha³, Alif Muhammad Mahrus⁴, Rafdhi Fatoni Septiyanto⁵, Aqilah Mutiara Nasaruddin⁶, Ahmad Alrifai⁷, Rizky Jauhari Angwarmase⁸

[#] Department of Statistics, Institut Teknologi Sepuluh Nopember, Sukolilo Campus, Surabaya, 60111, Indonesia
E-mail: 5003221079@student.its.ac.id

Abstract— High-Performance Concrete (HPC) is widely used in modern infrastructure due to its superior strength and durability, but predicting its compressive strength remains a complex task influenced by various material components and their nonlinear interactions. This study proposes the use of tree-based ensemble learning models—Random Forest, XGBoost, and LightGBM—to predict the compressive strength of HPC using a dataset of 1,030 concrete mix designs. Each model underwent standard preprocessing, 5-fold cross-validation, and extensive hyperparameter tuning to ensure robust evaluation. The performance was assessed using four regression metrics: RMSE, MAE, MAPE, and R-squared. Among the three models, LightGBM achieved the best performance with an RMSE of 4.034, MAE of 2.616, MAPE of 9.219, and R-squared of 0.945, indicating its high accuracy and generalization capability. Feature importance analysis using SHAP values revealed that the most influential variables were Age and Cement. In this finding, LightGBM proves to be a reliable and interpretable model for predicting concrete strength effectively.

Keywords—High-Performance Concrete; compressive strength; ensemble learning; SHAP; Feature Importance.

I. INTRODUCTION

Concrete has led to a high performance construction, bridge engineering, and infrastructure. While cement manufacturers possess extensive knowledge of cement, some may have a limited understanding of the specific requirements of concrete producers and contractors [1]. High-Performance Concrete (HPC), an advanced form of concrete, has evolved in its composition, properties, and adaptability to specific project needs.

It could be beneficial to provide some examples of common mix proportions for HPC. A key aspect of its production involves incorporating admixtures like fly ash, blast furnace slag, and silica fume, which significantly enhance its overall performance [2]. In reality, concrete mix proportions are not fixed; they change based on the specific characteristics of the raw materials used and the desired performance of the concrete in its application.

Determining the compressive strength of HPC is complex, influenced by many factors such as the type of aggregates (coarse or fine), water content, cement, and other additives, resulting in a complex relationship [3]. In certain situations, lowering the water ratio below a specific point is not practical from a mechanical perspective. This is because the strength of

the HPC will not significantly exceed the compressive strength of the aggregate. When the coarse aggregate becomes the limiting factor for compressive strength, the only way to achieve greater strength is to incorporate a stronger type of aggregate [4].

The construction industry highly values accurate predictions of concrete performance [5]. Machine learning methods effective at classifying data and making predictions by using historical information, which helps in understanding complex relationships within data. For predicting concrete strength, prominent machine learning methods include support vector machines, decision trees, artificial neural networks, and evolutionary algorithms [6].

Machine learning methods are well-suited to handle structured tabular data and offer advantages in terms of accuracy, feature importance analysis, and generalizability [7]. Machine learning could significantly reduce the need for time-consuming and resource-intensive physical testing. This research highlights the interdisciplinary relevance of statistical science and its potential contribution to sustainable development in the construction industry.

A previous study by [8] utilized the same dataset and applied an Artificial Neural Network (ANN) approach to predict the compressive strength of concrete. In that study, the ANN was trained using a pretraining technique, where initial weights

were derived from regression models such as Ridge Regression and Support Vector Regression. The goal was to improve prediction accuracy and accelerate the training process. While this approach effectively captured the nonlinear relationships between input and output variables, it still had limitations, such as sensitivity to initial parameters and the risk of overfitting without careful hyperparameter tuning.

In this study, an alternative approach is proposed using tree-based ensemble learning methods, namely Random Forest, XGBoost, and LightGBM. These methods are chosen for their ability to model complex nonlinear relationships without assuming any specific data distribution. They are also more robust to multicollinearity and outliers. Additionally, all three models provide feature importance metrics, which are valuable for interpreting the contribution of each input material to the final strength of the concrete.

Using the same dataset, this study aims not only to compare the performance of these three algorithms in predicting concrete compressive strength, but also to evaluate their effectiveness in terms of model interpretability and computational efficiency. The results of this research are expected to serve as a practical reference for civil engineering practitioners and researchers in selecting optimal and scalable predictive approaches for real-world applications.

II. MATERIAL AND METHOD

A. Literature Review

1) *Prediction Strength in US:* [8] conducted a seminal study titled “*Modeling of strength of high performance concrete using artificial neural networks*”, which laid the foundation for machine learning applications in civil engineering, particularly in predicting the compressive strength of concrete. The research utilized a comprehensive dataset obtained from the United States and employed Artificial Neural Networks (ANN) trained using a supervised regression technique. Initial model weights were derived from linear regression models such as Ridge Regression and Support Vector Regression (SVR) to capture the non-linear relationships between concrete mix proportions (e.g., cement, water, fly ash, coarse aggregate, etc.) and resulting strength. The model achieved high predictive accuracy, demonstrating ANN's ability to handle complex interactions among variables without requiring explicit physical formulations. Despite the advantages, Yeh's approach also faced limitations. The black-box nature of ANN reduced interpretability, and hyperparameter tuning required significant computational effort. Nonetheless, due to the structured nature and completeness of the dataset used, the UCI repository version of this dataset became widely adopted in benchmark testing across many machine learning studies. Its use promotes methodological consistency, allows fair comparison between algorithms, and helps validate novel techniques before applying them to local or customized datasets. Overall, Yeh's work played a crucial role in establishing machine learning as a viable alternative to empirical or statistical methods in predicting concrete properties.

2) *Prediction Strength Cement in UEA:* In a more recent study conducted in the United Arab Emirates, Huang et al. (2023) presented a paper titled “*Machine Learning Prediction of Compressive Strength of Concrete with Resistivity Enhancing Materials*” which extended traditional modeling

efforts by applying advanced algorithms on experimental data obtained from a hot-arid environment. The research focused on predicting the compressive strength of concrete containing special admixtures intended to improve resistivity and durability. The dataset comprised 680 concrete samples, each tested under varying curing ages, environmental exposures, and mix designs, reflecting construction practices typical in the UAE.

To assess model performance, the authors applied three machine learning algorithms—Support Vector Machine (SVM), Random Forest, and LightGBM. Evaluation was carried out using standard performance metrics including the coefficient of determination (R^2), root mean square error (RMSE), and mean absolute error (MAE). Among the models tested, Random Forest exhibited the best overall performance with an R^2 of 0.84, RMSE of 5.12 MPa, and MAE of 3.14 MPa. The findings underscored the ability of ensemble learning methods to model complex, non-linear relationships among materials and mechanical properties, especially under localized conditions. The study further emphasized the value of using regional datasets to develop models with greater generalizability and practical relevance in local construction environments.

Additionally, the study highlighted how materials such as resistivity-enhancing additives and curing techniques influence compressive strength, offering a refined understanding of mix optimization. By employing modern machine learning approaches alongside comprehensive lab-tested data, the authors demonstrated a methodological framework for future research and practical implementation in engineering design—particularly in regions with extreme climates like the UAE.

3) *Prediction of Compressive Strength of Concrete:* [9] conducted a study titled “*Machine Learning Prediction of Compressive Strength of Concrete with Resistivity Modification*”, focusing on the application of machine learning models to predict the compressive strength of concrete modified with resistivity-enhancing materials. The study utilized Support Vector Machine (SVM) and Random Forest (RF) models, trained on a dataset comprising 600 samples, which included parameters such as water-cement ratio, resistivity-modifying admixtures, curing age, and aggregate content.

Model performance was evaluated using metrics such as coefficient of determination (R^2), root mean square error (RMSE), and mean absolute error (MAE). The results indicated that the Random Forest model outperformed SVM, achieving an R^2 of 0.94, RMSE of 5.12 MPa, and MAE of 4.05 MPa on the testing set. The study highlighted the significant influence of resistivity-modifying admixtures and curing age on compressive strength, contributing approximately 85% to the model's predictive accuracy based on feature importance analysis. Additionally, the authors developed a predictive framework integrating resistivity measurements with machine learning, offering a novel approach for non-destructive strength assessment.

This study is highly relevant to the current research, as it demonstrates the efficacy of the Random Forest model, one of the models evaluated in this study, in predicting concrete compressive strength. The dataset used by Huang et al., which includes parameters like water-cement ratio and curing age,

aligns closely with the dataset employed in this research, reinforcing the importance of these variables in strength prediction. The superior performance of Random Forest over SVM provides a benchmark for comparing its performance against XGBoost and LightGBM in the current study. Furthermore, the use of R^2 , RMSE, and MAE as evaluation metrics ensures methodological consistency with this research.

B. Algorithm

1) *Random Forest*: Random Forest is a machine learning algorithm developed from the Classification and Regression Trees (CART) method by applying bootstrap aggregating (bagging) and random feature selection [10]. It uses a set of decision trees that form an ensemble of models, where each tree is trained using a random subset of different data. The basic concept of Random Forest involves predictor variables $X_1, X_2, \dots, X_p$ and response variable Y being simultaneously randomized from the feature space $\mathcal{X} \times \mathcal{Y}$, with an unknown joint distribution $P_{XY}(X,Y)$. Each tree in the ensemble is developed to find the optimal prediction function $f(X)$ to predict the value of Y .

Random Forest is a classifier that consists of a set of tree-structured classifiers $\{h(x, \Theta_k), k = 1, 2, \dots\}$, where $\{\Theta_k\}$ is an independent random vector that is equally distributed. Each tree votes for the most popular class at input x once during the classification process, and a majority voting mechanism is used to make the final decision. The average of each individual tree prediction is used to determine the final prediction for the regression case. Compared to using a single decision tree, this ensemble approach greatly increases prediction accuracy and decreases overfitting, producing more robust models and better generalization.

2) *eXtreme Gradient Boosting (XGBoost)*: [11] introduced a machine learning model named XGBoost to enhanced form of decision tree algorithms, utilizing and improved gradient boosting framework. The main core of the XGBoost algorithm is the decision tree, a supervised learning method used for both classification and regression. It divides data into smaller groups based on various threshold values of the input features.

XGBoost is an improvement upon the traditional Gradient Boosting (GB) algorithm, incorporating several additional features, such as regularization and tree pruning to help prevent overfitting. These enhancements offer robust and adaptable capabilities that make XGBoost a suitable choice among machine learning models. XGBoost is effectively suitable for sparse data because it incorporates a specialized method for finding the best splits in the data to efficiently manage missing or zero values [12].

XGBoost also recognized for its distinct objective function and its adaptability in selecting loss functions. Another advantage of XGBoost is quickly and efficiently handles large datasets. Moreover, XGBoost improves its accuracy through algorithmic enhancements [13]. Considering every possible tree structure is usually impractical. Instead, XGBoost employs a greedy algorithm: it starts with a single leaf and progressively adds branches to build the tree.

Similar to other data-driven techniques, XGBoost performance depends on the quality and quantity of the data. It can be effectively use for problems only when sufficient number of data is at disposal [2].

3) *LightGBM (Light Gradient Boosting Machine)*: is an efficient gradient boosting framework designed to handle large-scale tabular data with high computational efficiency. It utilizes a boosting approach to produce an optimal level of prediction accuracy [14]. Initially, the algorithm is configured with predefined parameters, such as the learning rate (`learning_rate`), and constructs an ensemble of decision trees, which are combined to generate a final prediction. Unlike traditional tree growth methods, LightGBM adopts a leaf-wise growth strategy, selecting splits based on the largest reduction in loss. This approach allows for deeper and more complex trees that can capture intricate patterns in the data. However, this increased complexity may lead to overfitting, especially when applied to small datasets. To address this, LightGBM incorporates regularization techniques to improve generalization performance.

A core strength of LightGBM lies in its histogram-based algorithm, which discretizes continuous features into bins to determine the optimal sample segmentation ratio. This significantly reduces memory usage and computation time [15], while accelerating the process of finding the best split points. In addition, LightGBM enhances training efficiency through Gradient-based One-Side Sampling (GOSS), which focuses on the most informative data instances with high gradients, and Exclusive Feature Bundling (EFB), which combines mutually exclusive features to reduce dimensionality without sacrificing predictive power. After the training process, LightGBM is evaluated using test data to assess the model's accuracy and overall performance. If the evaluation results are unsatisfactory, the model parameters can be fine-tuned to further optimize its performance. Considering its efficiency, flexibility, and scalability, LightGBM is regarded as a strong candidate for predicting concrete compressive strength, as it offers a balanced trade-off between predictive performance, interpretability, and computational efficiency.

C. Methods

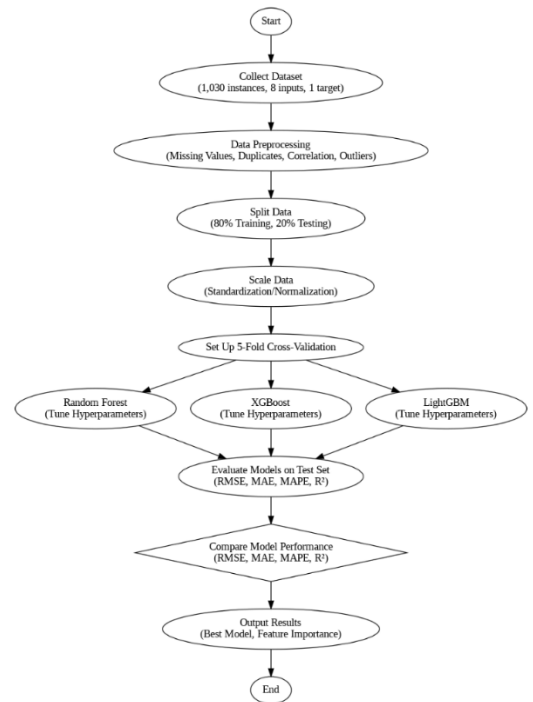


Fig. 1 Flowchart of the Machine Learning Workflow

The flowchart outlines the machine learning workflow for predicting the compressive strength of High-Performance Concrete (HPC). It begins with collecting a dataset (1,030 instances, 8 input features, 1 target), followed by preprocessing steps including handling missing values, duplicates, correlation analysis, and outlier detection. The data is then split (80% training, 20% testing), and scaling (standardization or normalization) is applied. A 5-fold cross-validation is set up, followed by training three models—Random Forest, XGBoost, and LightGBM—with tuned hyperparameters. The models are evaluated on the test set using RMSE, MAE, MAPE, and R^2 metrics. Model performance is compared based on these metrics, and the process concludes with outputting the best model and feature importance.

1) *Dataset Collection, Preprocessing, and Analysis:* The dataset used in this study, referred to as concrete_data, contains 1,030 instances of concrete mixtures, each characterized by eight input features and one target variable. The input features include the quantities of key components in the concrete mix, namely cement, blast furnace slag, fly ash, water, superplasticizer, coarse aggregate, fine aggregate, and age of the concrete. The target variable is the compressive strength of the concrete, measured in megapascals (MPa).

Table 1 Structure of Concrete Data

Y	X ₁	X ₂	...	X ₈
Y ₁	X _{1;1}	X _{2;1}	...	X _{8;1}
Y ₂	X _{1;2}	X _{2;2}	...	X _{8;2}
Y ₃	X _{1;3}	X _{2;3}	...	X _{8;3}
⋮	⋮	⋮	⋮	⋮
Y ₁₀₂₉	X _{1;1029}	X _{2;1029}	...	X _{8;1029}
Y ₁₀₃₀	X _{1;1030}	X _{2;1030}	...	X _{8;1030}

Table 2 Research Variables

Variable	Parameter	Unit
Y	Strength	MPa
X ₁	Cement	Kg/m ³
X ₂	Blast Furnace Slag	Kg/m ³
X ₃	Fly Ash	Kg/m ³
X ₄	Water	Kg/m ³
X ₅	Superplasticizer	Kg/m ³
X ₆	Coarse Aggregate	Kg/m ³
X ₇	Fine Aggregate	Kg/m ³
X ₈	Age	Kg/m ³

The preprocessing of the dataset begins with a preliminary inspection to assess data quality. No missing values were identified across the 1,030 instances and nine features (eight input features and one target variable), confirming the dataset's completeness.

To ensure data integrity, a duplicate detection process was carried out using pandas duplicated() function. A small number of duplicate data records were identified, which could potentially bias the model training process. To address this, duplicate entries were removed using the drop_duplicates() function, retaining only the first occurrence of each record. After this step, the dataset was re-inspected to verify that no duplicates remained.

The result shows that before the duplicate drop was performed, there were 25 duplicate data entries. After the duplicate data was handled, there were no more duplicate data entries detected.

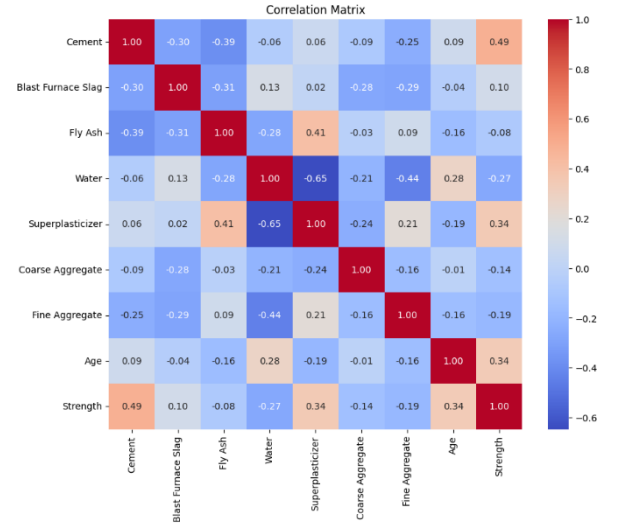


Fig. 2 Correlation Matrix

Following the preprocessing steps, a correlation analysis was performed to explore the relationships between the input features and the target variable, compressive strength. The correlation matrix, as illustrated in Fig. 2, provides valuable insights into the dependencies among the various components of the concrete mix. Notably, cement and compressive strength exhibit a moderate positive correlation of 0.49, suggesting that higher cement content tends to result in higher compressive strength. In contrast, the relationship between water and compressive strength is negative (-0.27), highlighting the inverse effect of water content on strength. Additionally, the negative correlation between fly ash and water (-0.65) implies that increasing the amount of fly ash reduces the required water content for the mix. The superplasticizer's correlation with water (-0.65) further reinforces this, as superplasticizers are typically used to maintain workability while reducing water content. While the age of the concrete shows a moderate positive correlation with strength (0.34), indicating that concrete continues to gain strength over time, the aggregates, both coarse and fine, show weak correlations with strength, suggesting their relatively lesser impact compared to the other materials. This correlation analysis serves as a foundation for selecting relevant features for the machine learning models and understanding the intricate relationships between the input variables in predicting concrete strength.

An outlier is an observation which deviates so much from other observations as to arouse suspicions that it was generated by a different mechanism [16]. In other words, outliers are data points that substantially differ from the general pattern or expected behavior within a dataset. While such deviations can sometimes distort analytical outcomes and warrant removal, in many cases they offer valuable insights, making their retention more appropriate.

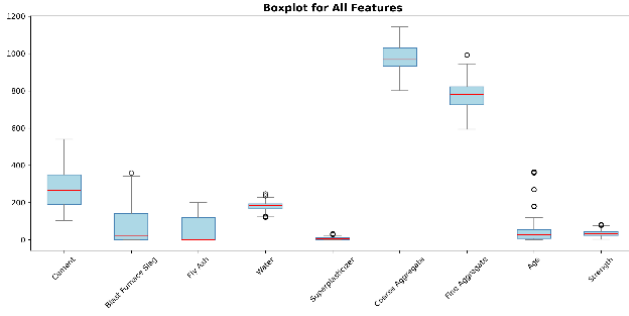


Fig. 3 Box Plot for All Features

Boxplot analysis of the concrete_data dataset shows the presence of outliers in features such as *Water*, *Superplasticizer*, *Fine Aggregate*, *Age*, and *Strength*. These outliers are considered valid experimental variations and are retained to preserve important information about uncommon mixture configurations.

The boxplot also reveals significant differences in feature value ranges, which can affect model training. To address this, Standard Scaling is applied by subtracting the mean and dividing by the standard deviation for each feature. This ensures that all features are on a comparable scale and prevents those with larger values from dominating the learning process. The structure of the data, including meaningful outliers, remains intact after scaling.

2) *Implementation Cross Validation in Data Training-Testing Ratios*: The dataset was partitioned into 80% training and 20% testing sets through a random split to avoid evaluation bias. This ratio provides a practical balance, offering sufficient data for model learning while retaining enough unseen data for objective testing. A random split was chosen because the dataset is non-time-series, meaning the observations are independent and not sequential, making random partitioning appropriate. The training set was used for model development and cross-validation, while the testing set was reserved for final performance evaluation.

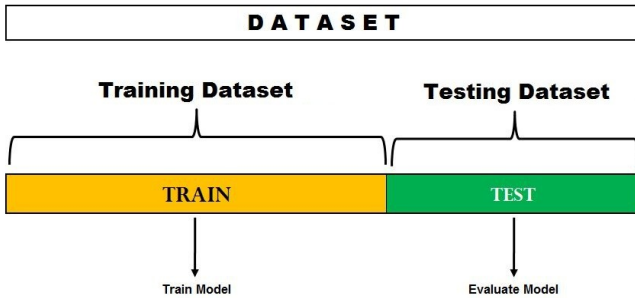


Fig. 4 Illustration of the Split Data Train and Test

To ensure a robust and unbiased estimation of model performance, a 5-fold cross-validation (CV) approach was employed throughout the modeling process. In the 5-fold CV process, the training data was partitioned into five equal-sized folds. During each iteration, four folds were used for training and the remaining fold was used for validation. This process was repeated five times such that each fold served once as the validation set. The results from each iteration were averaged to assess the model's generalization performance during training and guide the selection of hyperparameters. After the optimal model was obtained using the CV procedure, it was then

evaluated on the held-out test set (20% of the original data) to assess its predictive performance on unseen data. This two-stage evaluation process CV for model selection and separate test data for final evaluation helps prevent overfitting and provides a more realistic estimate of real-world performance.

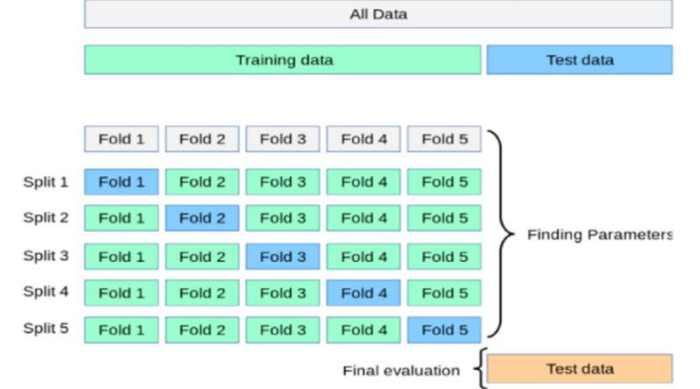


Fig. 5 Illustration of the 5-Fold Cross-Validation Procedure

This cross-validation framework was consistently applied to all three tree-based regression models evaluated in this study, namely Random Forest, XGBoost, and LightGBM. By using the same cross-validation scheme for each model, we ensured a fair comparison of their predictive capabilities under identical data conditions. Furthermore, this strategy minimizes the risk of overfitting to a single split of the data and improves the reliability of model selection.

3) *Hyperparameter Optimization*: Hyperparameter optimization was conducted to improve the predictive accuracy and stability of the Random Forest, XGBoost, and LightGBM models employed in this study. Since hyperparameters significantly influence the performance of tree-based ensemble methods, a systematic tuning process was implemented to identify parameter configurations that yield the best performance while reducing the risk of overfitting [17]. In this study, hyperparameter optimization was performed using Grid Search combined with five-fold cross-validation on the training dataset. This approach allowed the evaluation of various combinations of hyperparameter values systematically to identify the configuration with the best predictive performance based on the Root Mean Square Error (RMSE). Cross-validation ensured the robustness of the models and prevented information leakage during the tuning process [18]. For the Random Forest model, the tuned hyperparameters included the number of trees, maximum tree depth, minimum samples required for splitting a node, minimum samples required at a leaf node, and the maximum number of features considered for splitting [19]. For the XGBoost and LightGBM models, the tuned hyperparameters included the number of estimators, learning rate, maximum tree depth, subsample and colsample ratios, and regularization parameters [20], [21]. The best-performing hyperparameters obtained from this tuning process were then used to train the final models before evaluation on the test data.

4) *Model Evaluation Metrics*: To comprehensively evaluate the predictive performance of the models, this study employed Root Mean Square Error (RMSE), Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and the Coefficient of Determination (R^2) as evaluation indicators,

which are commonly used in regression tasks for predicting concrete compressive strength [22], [23].

The Root Mean Square Error (RMSE) measures the square root of the average squared differences between predicted and actual values, making it sensitive to large errors. The formula is as follows:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (1)$$

The Mean Absolute Error (MAE) calculates the average of the absolute differences between predicted and actual values, providing a straightforward interpretation of the average prediction error in the same units as the target variable:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2)$$

The Mean Absolute Percentage Error (MAPE) expresses the prediction error as a percentage, offering a scale-independent metric useful for practical interpretation:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|} \quad (3)$$

The Coefficient of Determination (R^2) indicates the proportion of variance in the actual values explained by the model, reflecting its goodness of fit:

$$MAPE = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4)$$

In this study, lower values of RMSE, MAE, and MAPE indicate better predictive accuracy, while higher R^2 values (closer to 1) demonstrate a stronger fit between the predicted and actual compressive strength values of concrete.

5) Feature Importance Using SHAP: In developing machine learning models, especially complex models such as ensemble trees (e.g., XGBoost) or deep learning, the “black box” problem often arises. This problem refers to a condition where the model is able to provide predictions with high accuracy, but the internal decision-making process is difficult or even impossible for humans to understand [24]. To overcome these challenges and increase transparency and trust in models, various Explainable Artificial Intelligence (XAI) techniques were developed. One of the most fundamental and trusted approaches in the XAI domain for measuring feature importance is SHAP (SHapley Additive exPlanations).

SHAP is a framework based on Shapley values, a concept from cooperative game theory that is mathematically optimal in distributing “payouts” (in this case, prediction results) among “players” (in this case, model features) [25]. Intuitively, SHAP calculates the contribution of each feature to a prediction by considering all possible combinations (coalitions) of features. This method answers an important question: “How much would the prediction output change if we knew the value of this feature, compared to if we did not know it, considering all other possible feature contexts?” This contribution is calculated as the average marginal contribution of a feature across all possible coalitions.

SHAP’s main advantage lies in its solid theoretical foundation, which guarantees three important properties: Local Accuracy, Missingness, and Consistency. Consistency ensures that if a model is changed so that the contribution of a feature

increases, its SHAP value will not decrease. This property overcomes the weaknesses of other methods such as LIME [26]. Local Accuracy ensures that the sum of the contributions of all features to one specific prediction will be exactly equal to the difference between that prediction and the model average prediction. This is formulated in an additive explanation model (a local linear model) as follows:

$$g(z') = \phi_0 + \sum_{i=1}^M \phi_i z'_i \quad (5)$$

In this equation, $g(z')$ is the model explanation for one prediction, where ϕ_0 is the base value or the average prediction of the model across the entire dataset. The variable M is the total number of features, and z' is the binary representation of the features in the coalition. The most crucial value is ϕ_i , which is the Shapley value for the i -th feature. This ϕ_i value represents the specific contribution of that feature to a single prediction, showing how much and in what direction (positively or negatively) the feature drives the prediction result from the baseline value of ϕ_0 .

While ϕ_i values provide a local explanation (for a single row of data), global feature importance can be obtained by aggregating these values across the entire dataset. This is done by calculating the average of the absolute SHAP values for each feature across all observations. The absolute value is taken because what matters in global ranking is the magnitude of the feature’s impact, not its direction (whether it increases or decreases the prediction). Thus, the global importance for feature j (I_j). This approach not only produces reliable feature rankings but also provides the basis for information-rich visualizations that can reveal complex patterns in model behavior.

III. RESULT AND DISCUSSION

The dataset used in this study consists of 1,030 observations of high-performance concrete mixtures, each described by eight numerical input features and one target output: compressive strength measured in MPa. To ensure unbiased model evaluation and robust training, several preprocessing and validation steps were carefully applied before fitting the models. As the data is non-temporal and independently distributed, it was first randomly partitioned into an 80% training set and a 20% testing set, as illustrated in Figure 4. The training set was exclusively used for model development and internal validation, while the testing set was reserved for final out-of-sample performance assessment, thereby preserving an unbiased estimate of model generalization.

Prior to model training, all input features underwent standard scaling using StandardScaler, which centered each feature to have a mean of zero and a standard deviation of one. Importantly, the scaling parameters were computed only from the training set and then applied to both training and testing data to prevent data leakage. This preprocessing step is crucial for ensuring that features with different units and scales contribute equally to model learning and also improves the convergence of gradient-based algorithms.

For robust model selection, the study employed 5-fold cross-validation within the training data. In each fold, models were trained on 80% of the data and validated on the remaining 20%, repeating this process five times with different validation splits. The average evaluation metrics across folds were then

used to guide hyperparameter tuning. This approach helps minimize overfitting, enhances model generalization, and ensures that each observation is used for both training and validation, reducing variance due to random sampling. Applying the same cross-validation procedure consistently across all models Random Forest, XGBoost, and LightGBM also ensures fair comparison under identical evaluation conditions.

To further improve performance, hyperparameter tuning was conducted using RandomizedSearchCV, which samples a fixed number of hyperparameter combinations from a predefined search space. Compared to exhaustive grid search, randomized search is computationally more efficient, especially when dealing with high-dimensional parameter spaces. Each sampled hyperparameter configuration was evaluated using 5-fold cross-validation on the training data, with negative mean squared error (neg-MSE) as the primary scoring metric, since scikit-learn's optimization framework seeks to maximize the chosen score. The configuration that achieved the lowest mean squared error during cross-validation was selected for final model training. The tuned hyperparameters included tree-related settings such as depth and number of estimators, regularization terms, feature sampling strategies, and learning rates. By evaluating 100 different hyperparameter combinations per model, the study ensured a thorough yet computationally feasible exploration of possible configurations.

A. Results of Random Forest Model

In this research, the concrete components (cement, water, blast furnace slag, superplasticizer, coarse, and fine aggregates and age) were identified as input variables for Random Forest Model. To achieve optimal performance and accuracy in these predictions, we optimized the Random Forest Model using Randomized-SearchCV, a method that conducting a cross-validated search over a defined range of parameter options [27].

For cross-validation, we used k-fold cross-validation. K-fold cross-validation involves training and testing a model K times, in each cycle, one fold of the data serves as the test set, while the remaining $K - 1$ folds are used for training. For our Random Forest model, we applied randomized search with 5-fold-cross-validation to find the best Random Forest optimal combination of parameters.

The critical hyperparameters that we tune are `n_estimators` (total iterations), `max_features` (number of features to be considered when searching for the best split at each node of tree), `max_depth` (maximum depth of each tree), `min_samples_split` (minimum number of samples required to split the internal node), `min_samples_leaf` (minimum number of samples that should be present in the leaf nodes), and `bootstrap` (determines whether bootstrap samples are used when building the tree).

In this research, the number of decision trees (or estimators) in the ensemble is 10, equally spaced numbers between 200 and 2000. The hyperparameters that determine the number of features to be considered when finding the best split at each tree node are `auto` (`max_features` = total number of features), `sqrt` (`max_features` = root of total number of features), and `log2` (`max_features` = log 2 of total number of features).

There is no limit to the maximum depth. The tree will fully grow until the minimum number of samples split or the

minimum samples of leaf satisfied. For minimum number of samples required to split, we tried different values (i.e. 2, 5, and 10) to explore the model and achieve optimal prediction accuracy. Similarly, we tried different values for minimum number of samples that should be present in the leaf nodes (i.e. 1, 2, and 4) to prevent overfitting.

When building the tree, we used bootstrap samples. Randomized-SearchCV will try 100 different hyperparameter combinations to be tested randomly. The model will be trained and evaluated 5 times (5 folds) for each hyperparameter combination.

Randomized-SearchCV always tries to maximize the scoring value. Since MSE is an evaluation metric (lower values are better), we used a “negative” version (negative MSE) for Randomized-SearchCV to maximize its score. In the end, the model with the smallest MSE (or largest negative MSE) will be chosen as the best model.

The best parameter found is the model with the number of decision trees (or estimators) in the ensemble is 1600, minimum number of samples required to split is 2, minimum number of samples that should be present in the leaf nodes is 1, the hyperparameters that determine the number of features to be considered when finding the best split at each tree node is `log2`, no limit to the maximum depth, and the entire dataset is used to train each tree for bootstrap.

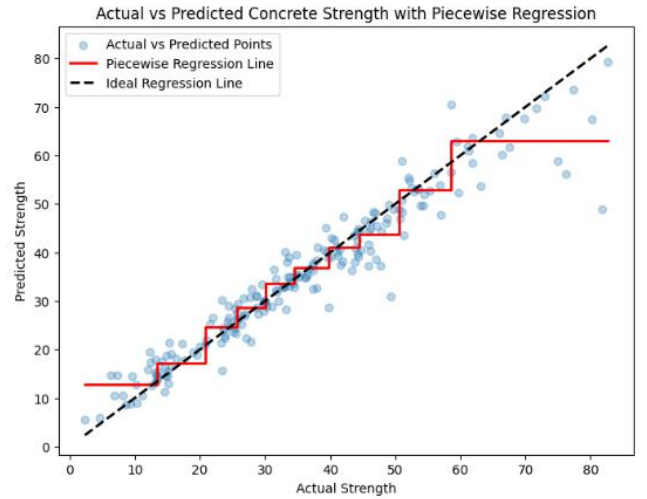


Fig. 6 Actual VS Predicted of Random Forest Model

To further illustrate the predictive performance of the optimized Random Forest model, an actual vs predicted plot with a piecewise regression line was generated in Fig 6. Each blue dot represents a pair of actual and predicted concrete strength values. Ideally, all blue dots will be at or near the black line that indicates a perfect prediction. The red line shows the model prediction trend in the form of horizontal segments.

Since Random Forest is a decision tree-based model, its predictions tend to be discrete, not linear, so the shape of the line is “ladder” or horizontal discontinuous line. This indicates that within a range of actual values, the model predicts the same or almost the same values, common in Random Forest due to its nature of clustering data based on leaf nodes. From Fig. 6, most of the blue dots are close to the ideal line, indicating that the prediction is quite accurate. The prediction results follow a pattern that is quite consistent with the actual data.

Table 4 Evaluation Metrics of Random Forest Model

Metrics	Training (Cross-Validation)	Testing
RMSE	5.234	4.877
MAE	3.584	3.034
MAPE	13.021	11.047
R-squared	0.891	0.920

From Table 4, the RMSE value of testing shows that the average prediction error is about 4.88 MPa. The RMSE value of testing is lower than training indicates good generalization (no overfitting). The average absolute error (MAE) in the testing data is only about 3.03 MPa. This shows that the model is quite accurate and stable. The relative error of the model (MAPE) is only about 11.05% on the testing data, which indicates that Random Forest model is highly accurate in predicting concrete strength.

The model is able to explain 92% of the variation in the testing data, indicating an accurate fit. The Random Forest model is quite fit at predicting concrete strength. The model is not overfitting or underfitting, since the testing performance is better than training. The accuracy of the model is quite high. The small RMSE, MAE, and MAPE values indicate that the predictions are close to the actual values. The R-squared value on testing is higher than training, indicating the model is not overfitting and is able to generalize data.

B. Results of XGBoost Model

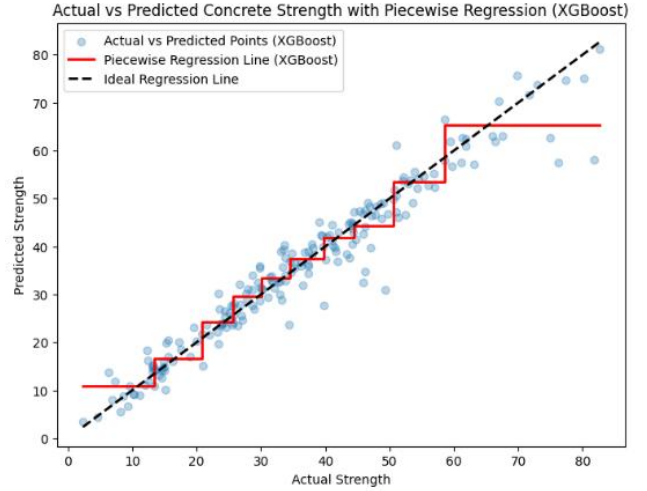
To develop and evaluate the XGBoost model, the dataset was divided into training and testing sets using an 80:20 split ratio. The training set, comprising 80% of the data, was used to fit the model, while the remaining 20% served as the test set for assessing the model's performance on unseen data. This split ensures that the model is trained on a substantial portion of the dataset while reserving a separate subset for unbiased evaluation of its predictive accuracy.

To ensure that all input features contribute equally to the XGBoost model, a StandardScaler was applied to the dataset. This preprocessing step standardizes the features by transforming them to have a mean of 0 and a standard deviation of 1. Standardization is particularly important for gradient-based algorithms like XGBoost, as it mitigates the influence of features with differing scales (e.g., cement content in kilograms versus age in days). By normalizing the feature distributions, StandardScaler enhances the model's convergence and predictive performance.

To robustly assess the model's generalization ability and prevent overfitting, 5-fold cross-validation (CV) was employed. In this approach, the training dataset was partitioned into five equal folds. For each iteration, one fold was reserved as the validation set, while the remaining four folds were used for training. This process was repeated five times, with each fold serving as the validation set exactly once. The average performance across all folds provided a reliable estimate of the model's predictive capability, ensuring that the results were not overly dependent on a specific subset of the data.

To optimize the XGBoost model's performance, RandomizedSearchCV was used to identify the best hyperparameter combination. This method randomly samples 100 combinations from a predefined hyperparameter grid and evaluates each using 5-fold cross-validation. The grid included key parameters such as `n_estimators` (100 to 1000), `learning_rate` (0.01 to 0.3), `max_depth` (3 to 10),

`min_child_weight` (1 to 7), `gamma` (0 to 0.4), `subsample` (0.6 to 1.0), `colsample_bytree` (0.6 to 1.0), `reg_alpha` (0 to 10), and `reg_lambda` (0 to 10). The objective was to minimize the negative mean squared error (`neg_mean_squared_error`), as RandomizedSearchCV maximizes the scoring metric, effectively selecting the model with the lowest mean squared error (MSE). This approach balances computational efficiency with thorough exploration of the hyperparameter space, yielding an optimized model with robust predictive performance.

**Fig. 7** Actual VS Predicted of XGBoost Model

To further illustrate the predictive performance of the optimized XGBoost model, an actual vs predicted plot with a piecewise regression line is presented in the figure. Each blue dot represents a pair of actual and predicted concrete strength values from the XGBoost model. Ideally, all blue dots would align with the black dashed line, which indicates a perfect prediction where actual and predicted values are equal. The red line depicts the model prediction trend in the form of horizontal segments, resembling a staircase pattern.

Since XGBoost is a tree-based ensemble model, its predictions tend to be discrete rather than continuous, resulting in a "ladder" or stepwise regression line. This indicates that within specific ranges of actual values, the model predicts relatively constant values, a characteristic influenced by the model's reliance on decision trees and binning of data. From the plot, most blue dots are distributed near the ideal line, suggesting that the predictions are reasonably accurate. The stepwise trend reflects a consistent pattern that aligns well with the actual data, demonstrating the model's ability to capture the underlying structure effectively.

Table 5 Evaluation Metrics of XGBoost Model

Metrics	Training (Cross-Validation)	Testing
RMSE	4.3506	4.2731
MAE	2.9647	2.7680
MAPE	10.2707	9.4170
R-squared	0.9256	0.9388

From the Table 5, the RMSE value for the testing set indicates an average prediction error of approximately 4.2731 MPa. The RMSE for testing is slightly lower than the cross-validated training value of 4.3506 MPa, suggesting good generalization and minimal overfitting. The average absolute error (MAE) on the testing data is about 2.7680 MPa, indicating

that the XGBoost model maintains high accuracy and stability across unseen data. The relative error (MAPE) on the testing set is approximately 9.4170%, reflecting the model's strong accuracy in predicting concrete strength.

The model explains about 93.88% of the variation in the testing data, as indicated by the R-squared value, demonstrating an excellent fit. The XGBoost model performs well in predicting concrete strength, with no significant overfitting or underfitting, as evidenced by the testing performance being comparable to the cross-validated training results. The low RMSE, MAE, and MAPE values confirm that the predictions are closely aligned with the actual values. The higher R-squared value on the testing set compared to the training set (0.9256) further supports the model's effective generalization to new data.

C. Results of LightGBM Model

In this LightGBM method, the critical hyperparameter that we tune are objective, `n_estimators`, `learning_rate`, `max_depth`, `num_leaves`, `min_child_samples`, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`.

Objective defines the loss functions to be tested, where `regression_l1` uses Mean Absolute Error (MAE) for L1 loss, `regression_l2` uses Mean Squared Error (MSE) for L2 loss, and `regression` defaults to MSE, allowing the model to optimize for different regression objectives. `n_estimators` (total iteration) specifies the number of boosting iterations (trees) to try, generating 10 evenly spaced integers from 100 to 1000 (e.g., 100, 200, ..., 1000) using numpy function, converted to integers for compatibility with LightGBM. `Learning_rate` in here is lists possible learning rates, where the possible values used in this analys (0.01, 0.05, 0.1, 0.2, 0.3). Learning rate control the step size of the gradient descent, with smaller values leading to slower but potentially more accurate learning. `max_depth` defines the maximum depth of each tree, where -1 indicates no depth limit, allowing the model to explore both shallow and deep trees.

`num_leaves` specifies the maximum number of leaves per tree, controlling model complexity, with higher values allowing more detailed splits. `min_child_samples` sets the minimum number of samples required in a leaf node, helping to prevent overfitting by ensuring leaves have sufficient data. `subsample` defines the fraction of training samples used for each tree, where values less than 1.0 introduce randomness to reduce overfitting. `colsample_bytree` specifies the fraction of features used for each tree, adding randomness to feature selection to improve generalization. `reg_alpha` sets the L1 regularization (Lasso) term to penalize large weights, helping to reduce model complexity and prevent overfitting. And also `reg_lambda` defines the L2 regularization (Ridge) term, similarly penalizing large weights to enhance model robustness.

Subsequently, the model was initialized by creating a LGBMRegressor with a fixed random state for reproducibility and parallel processing enabled (`n_jobs=-1`). To identify the best hyperparameter combination, we utilized RandomizedSearchCV with 100 iterations and cross-validation folds as defined in 5-fold cross-validation. The optimization criterion focused on minimizing the root mean squared error (RMSE) by specifying the scoring metric as negative mean squared error.

After fitting the random search on the training data, the best parameters were extracted and used to construct the final model. The best parameters found are included a subsample ratio of 1.0, indicating that the model used all available training data in each boosting iteration. The L2 regularization (`reg_lambda`) was set to 0.1, while L1 regularization (`reg_alpha`) was set to 1, which together helped mitigate overfitting by penalizing large coefficients. The learning objective selected was `regression_l2`, corresponding to the minimization of mean squared error. The optimal number of leaves was 10, reflecting a relatively conservative tree complexity to balance bias and variance. The number of boosting iterations (`n_estimators`) was set to 700, while the learning rate was configured at 0.1, which is a moderate value ensuring stable convergence. The model did not impose any limit on maximum tree depth (`max_depth=-1`), enabling trees to grow as needed to fit patterns in the data. And the column sampling ratio per tree (`colsample_bytree`) was set to 0.7, introducing additional randomness to improve generalization.

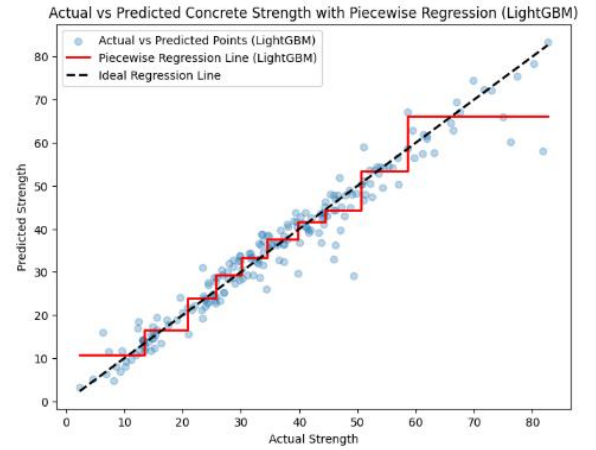


Fig. 8 Actual VS Predicted of LightGBM Model

To further illustrate the predictive performance of the optimized LightGBM model, an actual vs predicted plot with a piecewise regression line was generated in Fig.8. In this visualization, each observation is represented by a scatter point comparing actual concrete strength to predicted strength. The points are distributed closely around the diagonal ideal regression line, and the red piecewise regression line demonstrates that model predictions remain consistent across different ranges. Notably, slight underestimation appears at higher target values, though the deviation remains limited. This visual evidence aligns with the quantitative evaluation metrics, supporting the conclusion that the model achieves high accuracy and low bias.

While the scatter plot in Fig.8 illustrates the general alignment between observed and predicted values, a more precise evaluation requires examining standard of evaluation metrics. The main performance metrics obtained during cross-validation and testing are presented in Table 6.

Table 6 Evaluation Metrics of LightGBM Model

Metrics	Training (Cross-Validation)	Testing
RMSE	4.427	4.034
MAE	2.961	2.616
MAPE	10.151	9.219
R-squared	0.922	0.945

From Table 6, The Root Mean Squared Error (RMSE), which measures the average of prediction errors in squared terms, was 4.427 during cross-validation and slightly lower at 4.0343 on the test set. This indicates that the model was able to maintain or even improve its accuracy when predicting new or unseen data. The Mean Absolute Error (MAE), representing the average absolute difference between the predicted and actual values, also decreased from 2.961 in training to 2.6160 in testing, further confirming the model's consistency. The Mean Absolute Percentage Error (MAPE), which captures the average relative error expressed as a percentage, declined from 10.1516% to 9.2199%, suggesting that the model provided stable proportional predictions across datasets. Lastly, the R-squared value, reflecting the proportion of variance in the target variable explained by the model, increased from 0.9229 during training to 0.9454 on the test set. This improvement indicates that the model not only generalizes well but also captures most of the underlying patterns in the data.

The results of evaluation metrics in training and testing show that no significant discrepancy is observed. In fact, the test set shows slightly better performance across all metrics, with lower RMSE, MAE, and MAPE values, as well as a higher R-squared score. This pattern suggests that the model is not overfitting, since there is no marked difference in performance between the training and test sets. Likewise, there is no indication of underfitting, since the error rates are relatively low and the R-squared values are high on both datasets.

Therefore, the results indicate that the LightGBM model demonstrates a good balance between train and test, with no evident signs of underfitting or overfitting.

D. Optimal Model Selection and Performance Evaluation

In this subchapter, a comprehensive performance comparison is conducted among the three developed models Random Forest, XGBoost, and LightGBM to ascertain the most effective model for the task of predicting concrete compressive strength. The selection of the optimal model is rigorously based on its performance across four standard regression evaluation metrics, calculated from the test dataset: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and the Coefficient of Determination (R-square).

Table 7 Performance Metrics Comparison of Tree-Based Models

	Random Forest	XGBoost	LightGBM
RMSE	4.877	4.262	4.034
MAE	3.034	2.782	2.616
MAPE	11.047	9.642	9.219
R-squared	0.9202	0.9391	0.9454

The evaluation begins with an analysis of the model's prediction errors using MAE and RMSE. The Mean Absolute Error (MAE) represents the average of the absolute differences between the predicted and actual values, offering a direct and easily interpretable measure of the average error magnitude. The Root Mean Squared Error (RMSE), on the other hand, is calculated as the square root of the average of the squared differences. By squaring the errors before they are averaged, RMSE gives significantly more weight to larger prediction errors. A model that performs well on RMSE is one that does not make large, egregious errors. In both of these critical metrics, the LightGBM model demonstrated superior performance, achieving the lowest MAE of 2.615964 and the

lowest RMSE of 4.034344. This dual success indicates that LightGBM not only has the lowest average error but is also the most reliable in avoiding significant prediction deviations when compared to XGBoost and Random Forest.

To further assess the error from a relative perspective, the Mean Absolute Percentage Error (MAPE) was utilized. This metric provides a highly intuitive measure by expressing the average absolute error as a percentage of the actual values. Its unitless nature makes it excellent for comparing forecast accuracy across different models. Once again, the LightGBM model distinguished itself as the most accurate, recording the lowest MAPE of 9.219905%. This outstanding result signifies that, on average, the model's predictions deviate from the actual concrete strength by only 9.22%. This level of precision is markedly better than that of XGBoost (9.64%) and especially Random Forest, whose average percentage error exceeded 11%.

Finally, the Coefficient of Determination, or R-square, was assessed to measure the proportion of the variance in the dependent variable (concrete strength) that is predictable from the independent variables. A value closer to 1 indicates a better model fit and a higher degree of explanatory power. In this dimension, the LightGBM model also secured the top position with the highest R-square score of 0.945442. This means the model can successfully account for approximately 94.5% of the variability within the concrete strength data, confirming its robust ability to capture the underlying patterns. This explanatory power surpasses that of both XGBoost (93.9%) and Random Forest (92.0%).

In conclusion, after a thorough and multifaceted evaluation, the LightGBM model is definitively identified as the best model for this prediction task. The basis for this conclusion is its consistent and decisive superiority across every metric. It not only proved to be the most accurate model by yielding the lowest prediction errors in absolute (MAE, RMSE) and relative (MAPE) terms, but it also demonstrated the most powerful and robust model fit by achieving the highest R-square score. This comprehensive dominance validates LightGBM as the most reliable, precise, and powerful model, making it the clear choice for final implementation.

E. Features Importance

Based on the performance evaluation results from the test data, the LightGBM model showed the best performance compared to Random Forest and XGBoost. This model recorded the following evaluation values: RMSE of 4.034, MAE of 2.616, MAPE of 9.219, and R-squared of 0.94. The high R-squared value indicates that the model is able to explain 94% of the variability in concrete strength from the input variables used. Therefore, the feature importance analysis was focused on the LightGBM model.

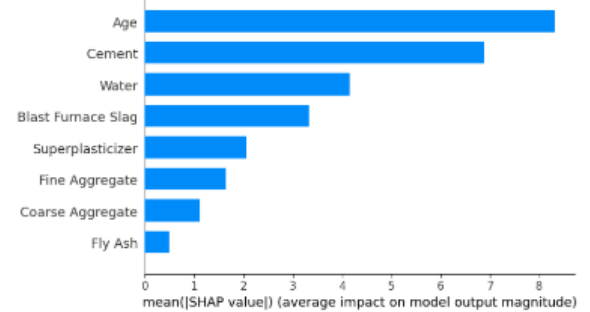


Fig. 9 SHAP-Based Feature Importance Analysis for LightGBM Model

To determine the contribution of a feature to model prediction, a study was conducted using SHAP (SHapley Additive exPlanations). A summary plot as absolute average SHAP values is shown in Figure 1, which shows the magnitude of the effect of each feature on the model output. The most significant feature is Age, followed by Cement, then Water and Blast Furnace Slag. The least significant features in prediction are Fly Ash and Coarse Aggregate.

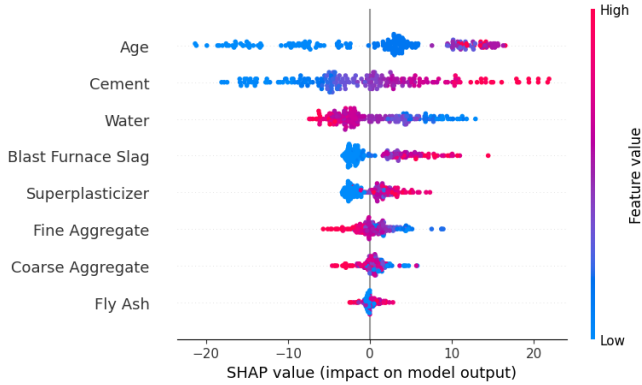


Fig. 10 Feature Contributions to Model Output Based on SHAP Values

Figure 10 depicts a beeswarm plot displaying the distribution of SHAP values of every feature, as well as the direction and magnitude of their contribution to the model result. Red shows high values of features, and blue shows low feature values. For instance, Age with high values (red) always has a positive contribution to concrete strength predictions, as expected by the behavior of concrete hardening as time passes. The same pattern can be seen for Cement, where more cement content increases concrete strength.

On the other hand, Water possesses more scattered SHAP value distribution and thus its role in concrete strength is situational and combination-dependent. Large values of Water even reduce the strength predictions in a manner in line with the axiomatic property that excessive water can weaken the structure of concrete.

With the application of the SHAP method, these results not only validate the stability of the LightGBM model as a prediction model but also provide an open interpretation of how each feature influences prediction outputs. The decomposition is useful for informing data-driven decision-making, particularly in concrete engineering.

IV. CONCLUSION

This study successfully utilized ensemble machine learning algorithms, including Random Forest, XGBoost, and LightGBM, to predict the compressive strength of High-Performance Concrete (HPC) using the *concrete_data* dataset with eight input features. Through comprehensive preprocessing, correlation analysis, scaling, and cross-validation, the three models were evaluated using RMSE, MAE, MAPE, and R-squared metrics to measure prediction accuracy. The results demonstrated that LightGBM achieved the best performance with an RMSE of 4.03, MAE of 2.62, MAPE of 9.22%, and an R-squared value of 0.945 on the testing data, outperforming XGBoost and Random Forest. This indicates that LightGBM can deliver accurate and stable predictions with good generalization capabilities for non-temporal HPC data.

Furthermore, the feature importance analysis using SHAP revealed that Age and Cement were the most significant factors influencing the compressive strength of concrete, followed by Water and Blast Furnace Slag, while Fly Ash and Coarse Aggregate contributed less to the prediction. These findings align with the theoretical understanding of HPC, where curing time and cement composition significantly affect strength development over time.

This study demonstrates that the application of machine learning in predicting the compressive strength of concrete can help reduce reliance on time-consuming and costly physical testing while contributing to the development of data-driven sustainable construction practices.

As a recommendation, future studies could consider utilizing local datasets under different climatic and material conditions and exploring deep learning or hybrid models to further improve prediction accuracy and interpretability in predicting the compressive strength of High-Performance Concrete under real-world conditions.

ACKNOWLEDGMENT

We extend our sincere thanks to Mr. Novri Suhermi, S.Si., M.Sc., for his exceptional guidance, unwavering support, and insightful feedback, all of which were crucial to the completion of this research. Our gratitude also goes out to all team members for their collaborative spirit and valuable contributions throughout this project. This work was conducted as part of our academic studies at the Department of Statistics, Institut Teknologi Sepuluh Nopember (ITS).

REFERENCES

- [1] A. Neville and P. C. Aitcin, "High Performance Concrete - An Overview," *Materials and Structures/Matériaux et Constructions*, pp. 111-114, 1998.
- [2] Y. Zhang, X. Yuan, X. Zhang, H. Wang, P. He, L. Luo and C. Xu, "Prediction of High-performance Concrete Compressive Strength using Decision Tree-guided Artificial Neural Network Pretraining Approach," *Engineering Applications of Artificial Intelligence*, vol. 156, no. A, pp. 1-15, 2025.
- [3] Z. Chen, Y. Feng, X. Zhang, X. Guo, L. Shao, Y. Cao, S. Li and L. Gao, "Similarity Criterion for The Nonlinear Thermal Analysis of The Soil Freezing Process: Considering The Dual Effect of Nonlinear Thermal Parameters and Boundary Conditions," *Acta Geotech*, vol. 17, no. 12, pp. 5709-5719, 2022.
- [4] P. C. Aitcin, "The Durability Characteristics of High Performance Concrete: A Review," *Cement and Concrete Composites*, vol. 25, no. 4-5, pp. 409-420, 2003.
- [5] P. K. Mehta and P. J. M. Monteiro, *Concrete: Microstructure, Properties, and Materials*, 4th Edition, McGraw-Hill Education, 2014.
- [6] M. M. Moein, A. Saradar, K. Rahmati, S. H. G. Mousavinejad, J. Bristow, V. Aramali and M. Karakouzian, *Predictive Models for Concrete Properties using Machine Learning and Deep Learning Approaches: A Review*, *J. Build. Eng.* 63, 105444, 2023.
- [7] I. B. Mustapha, M. Abdulkareem, T. M. Jassam, A. H. AlAteah, K. A. A. Soani, M. M. H. A. Tholaia, H. Nabus, S. C. Alih, Z. Abdulkareem and A. Ganiyu, "Comparative Analysis of Gradient-Boosting Ensembles for Estimation of Compressive Strength of Quaternary Blend Concrete," *International Journal of Concrete Structures and Materials*, vol. 18, no. 20, pp. 8-12, 2024.
- [8] I.-C. Yeh, "Modeling of strength of high-performance concrete using artificial neural networks," *Cement and Concrete Research*, vol. 28, no. 12, pp. 1797-1808, 1998.
- [9] C. Huang et al., "Machine Learning Prediction of Compressive Strength of Concrete with Resistivity Modification," *Materials Today Communication*, 2023.

- [10] Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [11] M. Niazkar, A. Menapace, B. Brentan, R. Piraei, D. Jimenez, P. Dhawan and M. Righetti, "Applications of XGBoost in Water Resources Engineering: A Systematic Literature Review (Dec 2018–May 2023)," *Environmental Modelling & Software*, vol. 174, pp. 1–6, 2024.
- [12] A. Chemura, D. Rwasoka, O. Mutanga, T. Dube and T. Mushore, "The Impact of Land-use/Land Cover Changes on Water Balance of The Heterogeneous Buzi Sub-cathment, Zimbabwe," *Remote Sens, Appl. Soc. Environ*, vol. 133, 2020.
- [13] H. Lu and X. Ma, "Hybrid Decision Tree-based Machine Learning Models for Short-term Water Quality Prediction," *Chemosphere*, vol. 249, 2020.
- [14] W. Liang, S. Luo, G. Zhao, and H. Wu, "Predicting Hard Rock Pillar Stability Using GBDT, XGBoost, and LightGBM Algorithms," *Mathematics*, vol. 8, no. 5, p. 765, May 2020, doi: 10.3390/math8050765.
- [15] Z.-P. Gao, Y.-J. Wang, H.-L. Lü, Q.-F. Li, C.-W. Shen, L. Liu, "Machine learning the nuclear mass", *Nucl. Sci. Tech.* 32 (2021) 118. <https://doi.org/10.1007/s41365-021-00958-z>.
- [16] D. M. Hawkins, "Identification of Outliers", vol. 11. New York, NY, USA: Springer, 1980.
- [17] P. Probst, M. N. Wright, and A. L. Boulesteix, "Hyperparameters and tuning strategies for random forest," *WIREs Data Mining and Knowledge Discovery*, vol. 9, no. 3, e1301, 2019.
- [18] M. Kuhn and K. Johnson, *Applied Predictive Modeling*. New York, NY, USA: Springer, 2013.
- [19] A. Liaw and M. Wiener, "Classification and regression by randomForest," *R News*, vol. 2, no. 3, pp. 18–22, 2002.
- [20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [21] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 3146–3154, 2017.
- [22] C. J. Willmott and K. Matsuura, "Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance," *Climate Research*, vol. 30, no. 1, pp. 79–82, 2005.
- [23] T. Chai and R. R. Draxler, "Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature," *Geoscientific Model Development*, vol. 7, no. 3, pp. 1247–1250, 2014.
- [24] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila and F. Herrera, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, no. October 2019, pp. 82–115, 2020.
- [25] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems*, Vols. 2017–December, no. Section 2, pp. 4766–4775, 2017.
- [26] C. Molnar, *Interpretable Machine Learning. A Guide for Making Black Box Models Explainable.*, Morrisville, NC, USA: Lulu Press, 2022.
- [27] N. Sharma, L. Malviya, A. Jadhav and P. Lalwani, "A Hybrid Deep Neural Net Learning Model for Predicted Coronary Heart Disease using Randomized Search Cross-Validation Optimization," *Decision Analytics Journal*, vol. 9, 2023.